

## The Real Reason Why Software Engineers Need Math

The effect of repetitive learning is nowhere more dramatic than in mathematics.



Software engineers often proclaim they never use any of the mathematics they learned in college. Come to that, they say they don't use much of the computer science they learned either. Having been a

mathematician for just over 30 years, I'll leave the CS part to others. But as far as the use of mathematics is concerned, let me admit up front the software engineers are dead right: they don't use their college mathematics.

They are also dead wrong. They make use of their college mathematics education every day.

There's no paradox here. It comes down to what is meant by the word "use." One meaning is the one those software engineers once encountered in their math classes. Having learned the rule for integration by parts in their calculus class, for example, they were then given exercises and exam questions requiring them to use that rule. This is the most familiar meaning attached to the word "use," and it's the one the engineers implicitly assume when they say they never use their college math. But it's a meaning built on what I call the "filling a vessel" view of the way humans learn.

According to the "filling a vessel" view, education consists largely of pouring facts into our brains; using what we have learned consists of pouring it back out. This is, dare I say it, a highly simplistic—and erroneous—view of edu-

cation. But it's one the education establishment (which I'm in) fosters every time it offers a course and then measures the results by setting a largely regurgitative, three-hour, written exam.

In contrast, all the evidence from several decades of research both into the way the brain works and into the learning process—and there are masses of such evidence—says the acquisition of facts and algorithmic procedures are merely surface manifestations of what goes on when people learn. (We know they are surface phenomena since we generally forget them soon after the last exam is over.) The real value of education is something else. Our brains are perhaps the world's best examples of an adaptive system. When we subject the human brain to an extended educational experience, it undergoes permanent changes. In physical terms, those changes are the growth and strengthening of certain neural pathways. In functional and experiential terms, we acquire new knowledge and skills. The more repetitive the learning process, the stronger and longer lasting are those changes.

The effect of repetitive learning is nowhere more dramatic than in mathematics. Formal mathematics is at most 5,000 years or so old. That's a mere blink of an eye in evolutionary time, and certainly not long enough for our brains to undergo any but the most minor changes. Thus, the mental processes we use to do mathematics must have been acquired and in use long before the Sumerians introduced abstract numbers some time between 8,000 and 5,000

years ago. The new twist required in order to do mathematics was to bring those capacities together and use them to reason not about the physical and social world for which they initially developed through natural selection, but rather a purely abstract world of the mind's own creation.

The human brain finds it extremely difficult to cope with a new level of abstraction. This is why it was well into the 18th century before mathematicians became comfortable dealing with zero

than most other people, in today's society the benefits affect everyone. For instance, a study carried out by the U.S. Department of Education in 1997 (the *Reilly Report*) showed students who complete a rigorous high school course in algebra or geometry do much better in terms of gaining entry to a college or university, and perform much better once they are there, whatever they choose to study. In other words, completion of a rigorous course in mathematics—it is not even

*The mental processes we use to do mathematics must have been acquired and in use long before the Sumerians introduced abstract numbers some time between 8,000 and 5,000 years ago.*

and with negative numbers, and why even today many people cannot accept the square root of minus-one as a genuine number.

But software engineering is all about abstraction. Every single concept, construct, and method is entirely abstract. Of course, it doesn't feel this way to most software engineers. But that's my point. The main benefit they got from the mathematics they learned in academia was the experience of rigorous reasoning with purely abstract objects and structures.

Moreover, mathematics was the only subject that gave them that experience. It's not what was taught in the mathematics class that was important; it's the fact that it was mathematical. In everyday life, familiarity breeds contempt. But when it comes to learning how to work in a highly abstract realm, familiarity breeds a sense of, well, familiarity—meaning that what once seemed abstract starts to feel concrete, and thus more manageable.

Though the payoff from learning (any) mathematics is greater for the computer professional

necessary the student does well in such a course—appears to be an excellent means of sharpening the mind and developing mental skills that are of general benefit.

(In my book, *The Math Gene: How Mathematical Thinking Evolved and Why Numbers Are Like Gossip* (Basic Books, 2000), I examine these ideas more carefully, and identify the survival advantages that led to the mental capacities required to do mathematics.)

As I have indicated in this short piece, the benefits for the software engineer are far greater. Indeed, they are an essential prerequisite. That's not usually given as the official reason for the obligatory math requirements for engineering students. But it is the main reason why they are beneficial. ■

---

**KEITH DEVLIN** (devlin@csl.stanford.edu) is the executive director of Stanford University's Center for the Study of Language and Information (CSLI).

---

© 2001 ACM 0002-0782/01/1000 \$5.00